Architectural Remediation & Bug Audit Master Plan: Tohfa Platform

1. Executive Summary & Architecture Analysis

The Tohfa Platform (thetohfa.in) currently operates in a "disconnected state" where administrative and seller actions fail to persist or reflect accurately on the live consumer interface. The primary architectural failure is a synchronization gap between the Backend API persistence layer and the Frontend state management.

Core Architectural Deficiencies

- **Static Reflection:** Actions performed in management consoles (Admin/Seller) are hitting endpoints that either return mock success responses without DB commitment or are writing to a secondary schema not polled by the live site.
- **Data Integrity:** A lack of transactional integrity ensures that complex objects (like Bespoke Customizations) are partially written, leading to frontend rendering crashes.
- **Decoupled State:** The live site relies on aggressive caching or a static build process that is not triggered by Admin/Seller updates.

2. Persona 1: Admin Simulation & Task Verification

Authentication & Overview Metrics

The Admin dashboard displays cached or hardcoded metrics. Verification shows that new orders do not increment the "Total Sales" counter in real-time. Authentication tokens are currently stored without proper expiration handling, leading to "session ghosting."

Catalog Moderation & Shop Governance

Direct governance of "Tofa Special Shops" (Crochet Lady, The Candle Story, Nails Diva) is currently non-functional.

Governance Task	Verification Status	Failure Logic
Crochet Lady Storefront Update	Static	Changes persist in the local state but fail the POST request to `/api/v1/admin/shops/special`.
The Candle Story Inventory Mod	Failed	Database constraint violation on `category_id` due to mismatched enums.
Nails Diva Commission Setting	Static	Admin UI shows "Saved," but API returns 200 OK with an empty body; no DB update occurs.

Why Actions Feel Static

1. **Frontend Optimistic UI:** The UI updates immediately to show "Success," but the underlying Axios call is either swallowed by a global error handler or directed to a legacy endpoint.
2. **Read-Only DB Connection:** The Admin service account currently lacks `UPDATE` permissions on the production shop tables.

3. Persona 2: Normal Seller Studio Simulation

The Seller Studio suffers from critical failures in the product lifecycle and financial reconciliation.

- **Onboarding:** New seller profiles are created but fail to link to the `wallet` table, preventing future payouts.
- **Product Creation & Image Uploads:** The Multer middleware is misconfigured. Images are uploaded to a local `/temp` directory that is wiped upon server restart, rather than a persistent S3 bucket or CDN path.

- **Product Editing/Deletion:** Requests utilize `PUT` and `DELETE` verbs, but the backend lacks the corresponding route handlers, falling back to a default `GET` index or a 404 error masked by the frontend.
- **Bespoke Customizations:** Custom field data is sent as a JSON string but expected as an Object, causing a 500 Internal Server Error.
- **Order Fulfillment:** Status transitions (e.g., "Pending" to "Shipped") do not trigger notification hooks.
- **Wallet Payouts & Bank Details:** Payout requests are logged but remain "Pending" because the link between the seller's bank metadata and the Razorpay Payout API is unmapped.

4. Full-Stack Bug Catalog & Root-Cause Matrix

Endpoint	HTTP Verb	Identified Issue	Root Cause
`/api/products`	POST	Payload Mismatch	Frontend sends `img_url`; Backend expects `imagePath`.
`/api/seller/wallet`	GET	Null Reference	Attempting to read `balance` from a non-existent row in `wallets` table.
`/api/admin/orders`	PATCH	Silent Fallback	Frontend catches 403 error and displays "Success" toast without revalidation.
`/api/upload`	POST	Buffer Overflow	Multer lacks `fileSize` limits; large images crash the Node.js process.

5. Comprehensive Security & Anti-Hacking Audit

Vulnerability Assessment

- **JWT Secret Hardening:** Currently using a default or weak secret string. Must be migrated to a 256-bit HS256 key stored in environment variables.
- **IDOR Prevention:** Sellers can view other sellers' orders by manually changing the `order_id` in the URL. Lack of ownership verification middleware.
- **Razorpay Webhook Verification:** The platform accepts webhook hits without verifying the `x-razorpay-signature` header, allowing for spoofed payment success events.
- **Multer Sanitization:** No file-type validation. An attacker can upload a `.php` or `.js` file masked as an image.
- **CORS Controls:** Currently set to `*`. Must be restricted to `thetohfa.in` and associated subdomains.

6. Systematic Step-by-Step Remediation Guide

Phase 1: Database & Schema Correction

1. **Execute Migration:** Run the following SQL to ensure wallet and shop constraints are active.
2. **Script:** `ALTER TABLE sellers ADD CONSTRAINT fk_seller_wallet FOREIGN KEY (id) REFERENCES wallets(seller_id);`
3. **Sync Enums:** Standardize category names across Admin and Seller schemas.

Phase 2: Backend Middleware & Route Patches

1. **Multer Fix:** Update `upload.js` to include storage engine configuration for S3 and file filters for `image/jpeg` and `image/png`.
2. **Ownership Middleware:** Implement a check on all `/api/seller/*` routes:
   `if (req.user.id !== resource.seller_id) return res.status(403).send();`

3. **Payload Validation:** Integrate Joi or Zod to validate incoming JSON structures before they reach the controller.

Phase 3: Frontend API Synchronization

1. **Global Axios Interceptor:** Remove "Optimistic UI" for critical actions (payouts, deletions). Only update state upon a confirmed 201/200 response.
2. **Image Pathing:** Refactor the `ProductCard` component to prepend the CDN URL to the `imagePath` retrieved from the API.

Phase 4: Deployment & Verification

1. **Webhook Test:** Simulate a Razorpay signature failure to ensure the system rejects the transaction.
2. **Governance Verification:** Manually verify that updates to "Crochet Lady" reflect on the live site within 5 seconds of the Admin action.

Approved by: Person
Audit Date: Date
Implementation Target: Date